

Scala Cheat Sheet — Examen

Sintaxis esencial: Variables, Tipos, Arrays, Funciones, Colecciones y Condicionales

1. Variables y Tipos Primitivos

Declaración explícita e implícita (Inferencia):

```
val a: Int = 10           // Immutable (Constante)
var b: Double = 5.5       // Mutable (Variable)
val nombre = "Ana"        // Infiere String
val flag = true           // Infiere Boolean
```

Tipos básicos (Value Types): `Int`, `Long`, `Double`, `Float`, `Boolean`, `Char`, `Unit` (equivalente a `void`, retorna `()`).

2. Arrays (¡Mejor con `val`!)

Usa `val` para que la referencia sea constante, aunque sus elementos internos sean mutables.

```
// 1. Por tamaño (inicializados a 0 / null)
val arr1 = new Array[Int](3)
arr1(0) = 10; arr1(1) = 20; arr1(2) = 30

// 2. Por literales (apply)
val arr2 = Array("A", "B", "C")

// 3. Array mixto polimórfico
val mixed = new Array[Any](2)
mixed(0) = "texto"; mixed(1) = 99

// Recorrido rápido con bucle for
for (i <- 0 until arr1.length) {
  println(arr1(i))
}
```

3. Funciones (Sintaxis `def`)

La última expresión evaluada es el valor de retorno (no hace falta `return`).

```
// Función estándar con tipo de retorno explícito
def suma(x: Int, y: Int): Int = {
  val res = x + y
  res // Retorno implícito
}

// Función de una sola línea (omite llaves)
def esMayor(edad: Int): Boolean = edad >= 18

// Procedimiento (devuelve Unit / void)
def saludar(nombre: String): Unit = {
  println("Hola " + nombre)
}
```

4. Estructuras de Control (If / Else)

En Scala, las estructuras `if/else` devuelven un valor (como el operador ternario en otros lenguajes).

```
val x = 10
val resultado = if (x > 5) "Mayor" else "Menor"

// If-else clásico
if (x == 10) {
  println("Diez")
} else {
  println("Otro")
}
```

5. Listas y Tuplas (Inmutables)

Listas (`List`): Inmutables y enlazadas.

```
val lista1 = List(1, 2, 3)
val lista2 = 0 :: lista1 // Prepend (añade al inicio: List(0, ...))
val listaConcat = lista1 ::: lista2 // Concatenación
```

Tuplas: Agrupan elementos de distintos tipos.

```
val persona = ("Juan", 25, true)
val nombre = persona._1 // Acceso por índice basado en 1
```

6. Patrones y Bucles `for`

Pattern Matching (`match`):

```
val i = 2
i match {
  case 1 => println("Uno")
  case 2 => println("Dos")
  case _ => println("Otro")
}
```

Bucles y Generadores:

```
// Recorrer rango
for (i <- 1 to 5) { print(i) } // 1,2,3,4,5
for (i <- 1 until 5) { print(i) } // 1,2,3,4
```